PROTOFIRE

October 2026

Smart contract audit
rain.datacontract

Content

Project description **3**
Executive summary **3**
Reviews **3**
Security findings **6**
**** Critical 6
*** High 6
** Medium 6
M01 - Silent Failure in writeZoltu When Proxy Is Missing 6
* Low 6
L01 - Off-by-One Memory Read in write Function 6
Informational 7
I01 - Stale or Misleading Assembly Comments 7
I02 - Hardcoded Deterministic Deployment Proxy Address 8
General Risks **9**
Approach and methodology **10**

Project description

This repository contains a Solidity library titled LibDataContract, which is a simplified, gas-optimized reimplementation of SSTORE2. It builds creation code that stores arbitrary data in contract bytecode (`contractCreationCode`) and reads that data back (`read`, `readSlice`). The library itself does not deploy anything: deployment (direct `create`/`create2`, a deterministic deployment proxy, etc.) is left to the caller. It features assembly optimizations for minimal gas usage, strict bounds checks on slice reads. Building creation code requires an EVM with `MCOPY` (Cancun); reads do not.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.datacontract

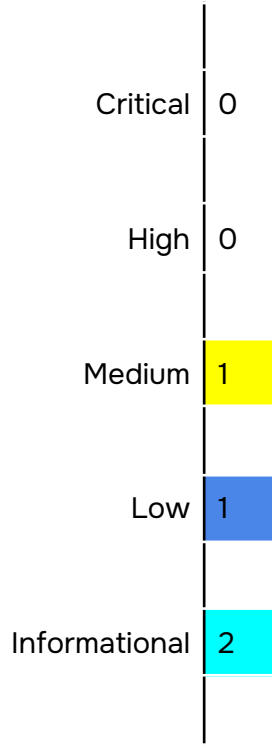
Reviews

Date	Repository	Commit
26/01/26	rain.datacontract	061bf7cd63496ccbe4f74bcd0c48715883c1c3cb
28/01/26	rain.datacontract	6217921940ff7b2f0bd260d79573b109a2f8bf66
09/02/26	rain.datacontract	3bff30782e408386f73b9353c5122e2e0ab24df2
25/08/26	rain.datacontract	c8f3188848c3ff0935fa5714b29022bd4c1f63aa **tag sol-v0.1.3**
08/10/26	rain.datacontract	6f939def9f7fbd10875f094456ea27bd8fbd277a **tag sol-v0.2.0**

Scope

Contracts
src/lib/LibDataContract.sol
src/error/ErrDataContract.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

M01 - Silent Failure in writeZoltu When Proxy Is Missing

The `writeZoltu()` function performs a low-level call to a hardcoded deterministic deployment proxy address (0x7A0D...). It relies on the success boolean returned by the call instruction to detect failures.

However, in the EVM, a low-level call to an address that contains no code (i.e., if the proxy is not deployed on the current chain) returns true (success) but executes nothing and returns no data. In this scenario:

1. The call returns success = true.
2. The mload(0) reads 0 (since no data was returned).
3. The if (!success) check passes.
4. The function returns address(0) effectively silently failing.

This differs from the standard write function, which explicitly checks if (pointer == address(0)) to detect deployment failures.

Path: src/lib/LibDataContract.sol

Recommendation: Update the validation logic in writeZoltu to require that the returned address is non-zero. This ensures the deployment actually occurred.

Found in: 061bf7cd

Status: Fixed at 94159c97

* Low

L01 - Off-by-One Memory Read in write Function

The `write()` and `writeZoltu()` functions calculate the payload size for the create/call opcodes by adding `prefixLength` (13) to the data length stored in the prefix.

The issue arises from "double counting" the 0x00 separator byte:

1. Physically: The 0x00 byte is already included within the 13-byte prefixLength.
2. Logically: The length stored in the prefix (calculated as user_data_length + 1 in newContainer) also accounts for this 0x00 byte to satisfy the contract deployment logic.

When these are added together (prefixLength + storedLength), the 0x00 byte is effectively counted twice, causing the calculated size to be 1 byte larger than the actual populated memory.

As a result, the create opcode reads one byte past the end of the valid data. While Solidity's 32-byte memory alignment ensures this extra byte is read from zero-padded reserved memory (preventing dirty reads), the logic is technically incorrect and consumes a negligible amount of extra gas.

Path: src/lib/LibDataContract.sol

Recommendation: Subtract 1 from the size calculation in write and writeZoltu to correct the double counting.

Found in: 061bf7cd

Status: Fixed at d8b8fa3b

Informational

I01 - Stale or Misleading Assembly Comments

The documentation block above the `BASE_PREFIX` constant details an assembly sequence based on the original `SSTORE2` implementation. It lists opcodes, such as 0x63 (PUSH4), and offsets, like 0x0e. However, these do not match the actual optimized bytecode implemented in `BASE_PREFIX`, which uses 0x61 (PUSH2) and an offset of 0x0c.

While the comments include a note explaining the optimization ("Note also that we don't need 4 bytes..."), the primary visual opcode map contradicts the code immediately following it. This can confuse developers or auditors trying to verify the bytecode construction manually.

Path: src/lib/LibDataContract.sol

Recommendation: Update the opcode breakdown in the comments to exactly match the values in `BASE_PREFIX` (showing 0x61 and 0x600c). If the comparison to the

original SSTORE2 implementation is important for context, clearly label the existing chart as "Reference Implementation" and add a corrected chart labeled "Current Implementation."

Found in: 061bf7cd

Status: Fixed at 00b0fa01

I02 - Hardcoded Deterministic Deployment Proxy Address

The `writeZoltu()` function performs a low-level call to a hardcoded deterministic deployment proxy address: `0x7A0D94F55792C434d74a40883C6ed8545E406D12`.
This address is embedded directly in the function body and cannot be modified or configured.
Hardcoding an external dependency inline inside function logic introduces some maintainability and integration risks:

- Critical configuration is hidden inside implementation details,
- The dependency may be easy to overlook during reviews and integrations.

Path: src/lib/LibDataContract.sol

Recommendation: Move the proxy address into a named constant variable with clear documentation.

Found in: 061bf7cd

Status: Fixed at 18eb6419

General Risks And Assumptions

- Contract Size Limit (EIP-170): The Ethereum Virtual Machine (EVM) imposes a limit on contract size (typically 24KB). Attempting to write a data blob that, combined with the deployment prefix, exceeds this limit will cause the write transaction to fail. Users must ensure their data fits within these constraints.

- Immutability: Data stored via this method is immutable. Unlike standard storage slots which can be updated, updating data stored in bytecode requires deploying a completely new contract and updating any references to it in the consuming system.

- Deployment Cost: While reading this data is gas-efficient, the initial cost to deploy the data contract scales linearly with the size of the data. Large datasets may incur significant gas costs during the write operation.

- Cancun Requirement: `contractCreationCode` uses `MCOPY`. On pre-Cancun chains it halts with an invalid opcode at runtime, even if the consumer compiled with a Cancun evm_version. Reads have no such requirement.

- `readSlice()` truncates start/length to 16 Bits: Since sol-v0.2.0, `readSlice()` takes `uint256` start and `uint256 length` and, as documented in its NatSpec, uses only their low 16 bits (`and(x, 0xffff)`). The bounds check runs on the truncated values, so values of 65536 or more wrap around instead of reverting: `readSlice(ptr, 65536, 4)` on a 100-byte data contract returns bytes `[0, 4)`, and `readSlice(ptr, 0, 65536)` returns an empty `bytes`. The library stays memory-safe, because allocation and `extcodecopy` both use the truncated length. Integrators that derive offsets or lengths from untrusted or computed input must check that they are within `[0, 65535]` (and within the data length) before calling, and must not assume `result.length == length` for unchecked input. Note that the README bullet "Reverts on out-of-range slices instead of silently truncating" applies only to values within 16 bits.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.